

Java 1.4 — assert Keyword

Java Version: 1.4

Year: 2002

Feature: assert keyword

UniqueClassName: AssertKeyword

1. Definition

The Java **assert keyword** was introduced in **Java 1.4** to help developers detect programming errors by checking assumptions made by the program during development and testing.

An assertion evaluates a **boolean expression**:

- If the expression is true, execution continues normally.
- If the expression is false, Java throws an `AssertionError`.

Syntax

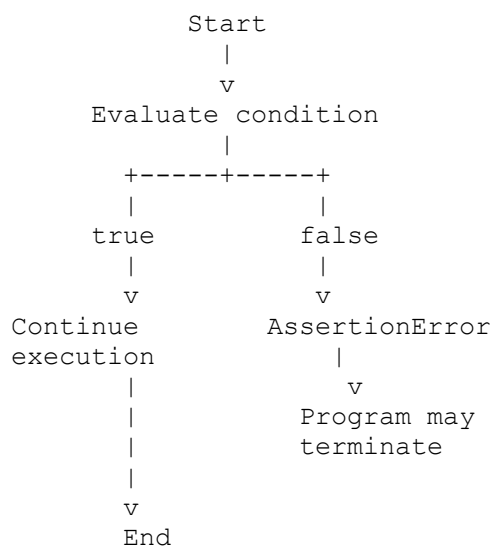
`assert condition;`

or

`assert condition : errorMessage;`

The second form provides additional information when the assertion fails.

2. Assertion Flowchart



3. Basic Java Code Example

```
public class AssertKeyword {  
  
    public static void main(String[] args) {  
  
        int age = 25;  
  
        assert age >= 18;  
  
        System.out.println("Age is valid.");  
    }  
}
```

Execution

```
age = 25  
  |  
  v  
age >= 18 ?  
  |  
  +----- true -----> Continue  
                        |  
                        v  
                    "Age is valid."
```

4. Assertion Failure Example

```
public class AssertKeyword {  
  
    public static void main(String[] args) {  
  
        int age = 15;  
  
        assert age >= 18 : "Age must be at least 18";  
  
        System.out.println("Age is valid.");  
    }  
}
```

If assertions are enabled, the condition:

`age >= 18`

is false, so Java throws:

`java.lang.AssertionError: Age must be at least 18`

Flow

```
age = 15
  |
  v
age >= 18 ?
  |
  v
false
  |
  v
AssertionError
  |
  v
"Age must be at least 18"
```

5. How to Enable Assertions

Assertions are **disabled by default** when running a Java application.

Enable them using the `-ea` or `-enableassertions` JVM option:

```
java -ea AssertKeyword
```

For example:

```
javac AssertKeyword.java
java -ea AssertKeyword
```

Without `-ea`:

```
java AssertKeyword
```

the assertion is normally not executed.

6. Assertion with a Method

```
public class AssertKeyword {

    static int calculateSquare(int number) {

        assert number >= 0 : "Number cannot be negative";

        return number * number;

    }

}
```

```

public static void main(String[] args) {

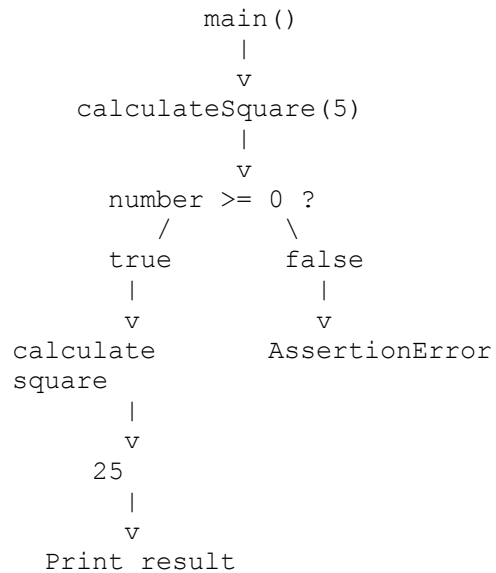
    int result = calculateSquare(5);

    System.out.println("Result: " + result);

}
}

```

Flowchart



7. assert vs if

assert	if
Primarily for development/testing	Used for normal application logic
Can be disabled	Always executed
Failure produces <code>AssertionError</code>	Can explicitly throw/handle an exception
Checks programmer assumptions	Handles expected runtime conditions
Enabled using <code>-ea</code>	No JVM option required

Example

Assertion:

```
assert value >= 0 : "Value must be positive";
```

Normal validation:

```
if (value < 0) {  
    throw new IllegalArgumentException(  
        "Value must be positive"  
    );  
}
```

Important: Do not use assert for validation that must happen in production, because assertions can be disabled.

8. Key Points

1. assert was introduced in **Java 1.4**.
2. It checks assumptions made by the programmer.
3. It evaluates a boolean condition.
4. true → execution continues.
5. false → AssertionError is thrown when assertions are enabled.
6. Assertions are disabled by default.
7. Use -ea to enable them.
8. Assertions are primarily useful for **debugging, development, and testing**.
9. They should not replace mandatory runtime validation.

In one line

assert = "I expect this condition to be true; if it isn't, something is wrong with the program."

Java 1.4 — Regular Expressions (Regex API)

Java Version: 1.4

Year: 2002

Feature: Regular Expressions (Regex)

UniqueClassName: RegexAPI

1. Definition

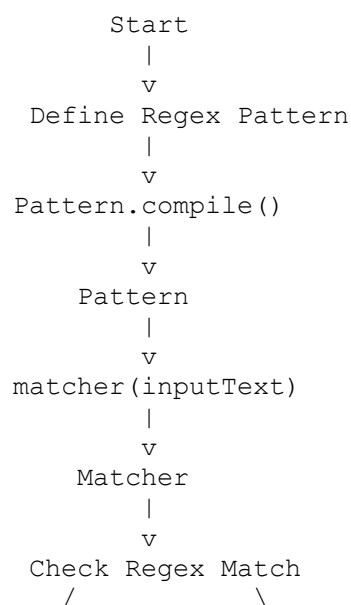
Regular Expression (Regex) is a pattern-matching mechanism used to **search, identify, validate, split, and replace text** according to a specified pattern.

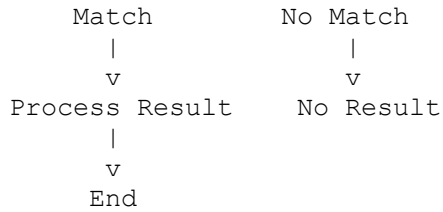
Java 1.4 introduced the `java.util.regex` package, providing built-in support for regular expressions.

The main classes are:

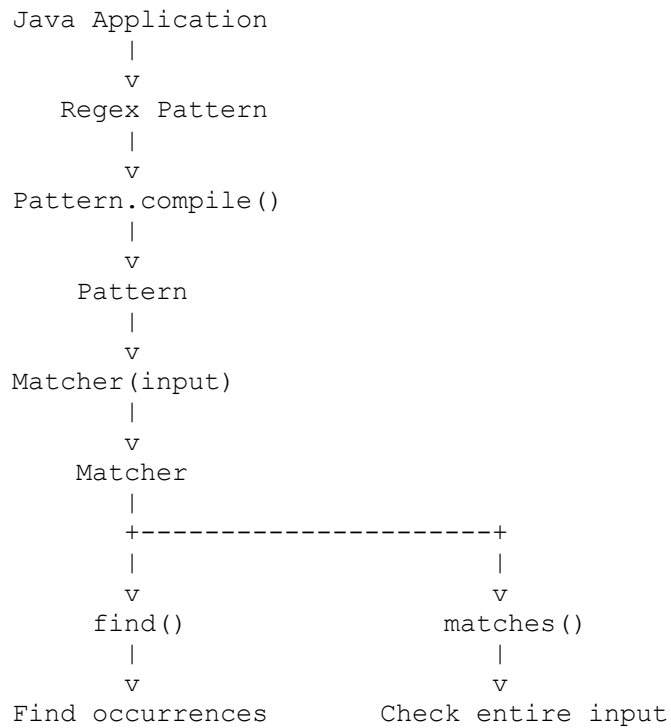
Class / Interface	Purpose
Pattern	Compiles a regular expression into a reusable pattern
Matcher	Performs matching operations against input text
PatternSyntaxException	Reports invalid regular-expression syntax

2. Basic Regex Flowchart





3. Java Regex Architecture



4. Basic Java Code Example

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class RegexAPI {

    public static void main(String[] args) {

        String text = "Java is powerful. Java is popular.";

        // Define the regular expression
        String regex = "Java";

        // Compile the pattern
        Pattern pattern = Pattern.compile(regex);
```

```

// Create a matcher
Matcher matcher = pattern.matcher(text);

// Find all occurrences
while (matcher.find()) {

    System.out.println(
        "Match found: " +
        matcher.group()
    );

    System.out.println(
        "Start: " +
        matcher.start()
    );

    System.out.println(
        "End: " +
        matcher.end()
    );

}
}
}

```

Output

```

Match found:    Java
Start:         0
End:           4

```

```

Match found:    Java
Start:         19
End:           23

```

5. Regex Matching Flow

For:

```
String text = "Java is powerful. Java is popular.";
```

and:

```
String regex = "Java";
```

the execution is:

```

"Java"
|

```

```

      v
Pattern.compile()
      |
      v
      Pattern
      |
      v
matcher(text)
      |
      v
      Matcher
      |
      v
matcher.find()
      |
+-----+-----+
|               |
Found          Not Found
|
v
matcher.group()
|
v
"Java"

```

6. Regex Validation Example

Regex can also validate formats.

For example, checking whether a string contains only digits:

```

import java.util.regex.Pattern;

public class RegexAPI {

    public static void main(String[] args) {

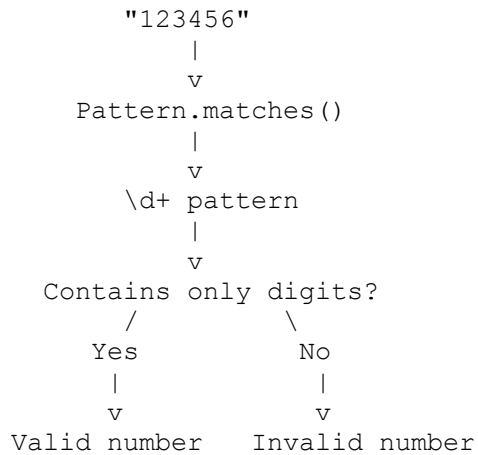
        String value = "123456";

        boolean valid =
            Pattern.matches("\\d+", value);

        if (valid) {
            System.out.println("Valid number");
        } else {
            System.out.println("Invalid number");
        }
    }
}

```

Flowchart



7. Common Regex Symbols

Regex	Meaning	Example
.	Any character	a.c
\\d	Digit	123
\\D	Non-digit	abc
\\w	Word character	Java1
\\s	Whitespace	Space/tab
+	One or more	a+
*	Zero or more	a*
?	Zero or one	a?
{n}	Exactly n times	\\d{4}
{n,m}	n to m times	\\d{2,4}
^	Beginning	^Java
\$	End	Java\$
[abc]	a, b, or c	[aeiou]
[^abc]	Not a, b, or c	[^0-9]

`	`	OR
0	Capturing group	(Java)

8. Replace Text Using Regex

The Matcher class can also replace matching text.

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class RegexAPI {

    public static void main(String[] args) {

        String text =
            "Java is easy. Java is powerful.";

        Pattern pattern =
            Pattern.compile("Java");

        Matcher matcher =
            pattern.matcher(text);

        String result =
            matcher.replaceAll("JAVA");

        System.out.println(result);
    }
}
```

Output

JAVA is easy. JAVA is powerful.

Replacement flow

Original Text

|

v

Create Pattern

|

v

Create Matcher

|

v

replaceAll()

|

v
Replace matching text
|
v
Modified Text

9. Important Pattern and Matcher Methods

Pattern

Pattern.compile()
Pattern.matches()

Matcher

matcher.find()
matcher.matches()
matcher.group()
matcher.start()
matcher.end()
matcher.replaceAll()
matcher.replaceFirst()

10. Key Points

- **Java 1.4** introduced the standard java.util.regex API.
- Pattern represents a compiled regular expression.
- Matcher performs matching against input text.
- Regex can be used for **searching, validation, extraction, splitting, and replacement**.
- PatternSyntaxException is thrown when the regex syntax is invalid.
- Regex is widely used for validating **emails, phone numbers, dates, IDs, passwords, URLs, and text patterns**.

In one line

Regex API = Pattern + Matcher → powerful pattern-based text searching, validation, extraction, and replacement.

Java 1.4 — Logging API

Java Version: 1.4

Year: 2002

Feature: Logging API

UniqueClassName: LoggingAPI

1. Definition

The **Java Logging API** is a standard Java API introduced in **Java 1.4** for recording application events, diagnostic information, warnings, errors, and debugging messages.

It is provided through the:

`java.util.logging`

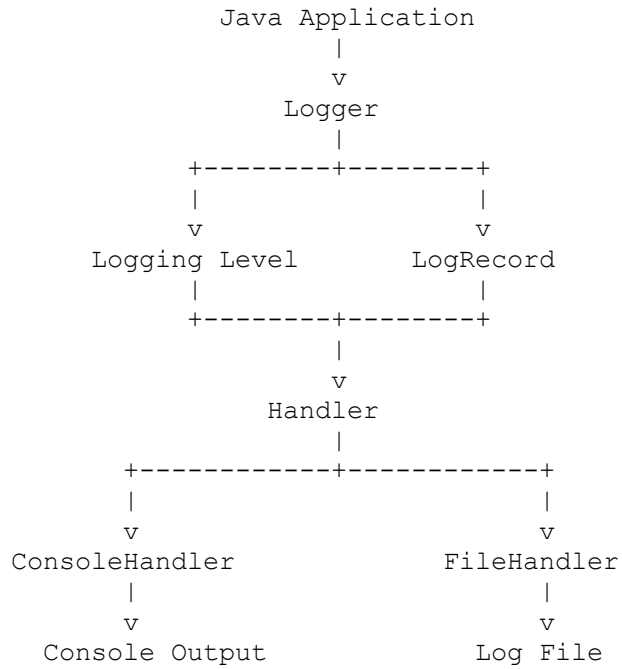
package.

The Logging API allows developers to record messages at different severity levels instead of using `System.out.println()` throughout an application.

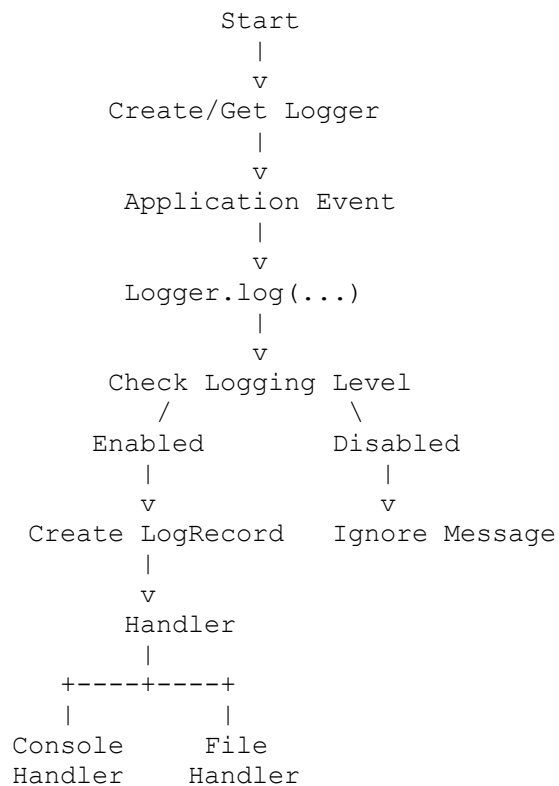
Main logging levels

Level	Purpose
SEVERE	Serious failure or error
WARNING	Potential problem
INFO	General application information
CONFIG	Configuration information
FINE	Detailed debugging information
FINER	More detailed debugging information
FINEST	Most detailed debugging information

2. Logging API Architecture



3. Logging Flowchart



v	v
Console	Log File

4. Basic Java Code Example

```
import java.util.logging.Logger;

public class LoggingAPI {

    private static final Logger logger =
        Logger.getLogger(LoggingAPI.class.getName());

    public static void main(String[] args) {

        logger.info("Application started.");

        logger.warning("This is a warning message.");

        logger.severe("This is a severe error message.");

        System.out.println("Application execution completed.");

    }

}
```

Example output

```
INFO:           Application started.
WARNING:        This is a warning message.
SEVERE:         This is a severe error message.
Application execution completed.
```

The exact output format can vary depending on the JVM and logging configuration.

5. Logging Levels Example

```
import java.util.logging.Logger;

public class LoggingAPI {

    private static final Logger logger =
        Logger.getLogger(LoggingAPI.class.getName());

    public static void main(String[] args) {

        logger.severe("SEVERE message");
        logger.warning("WARNING message");
        logger.info("INFO message");

    }

}
```

```

        logger.config("CONFIG message");
        logger.fine("FINE message");
        logger.finer("FINER message");
        logger.finest("FINEST message");
    }
}

```

By default, the console handler commonly displays messages from INFO and above, so lower-level messages such as FINE, FINER, and FINEST may not appear unless logging is configured to allow them.

6. Logging to a File

The `FileHandler` can write logging information to a file.

```

import java.io.IOException;
import java.util.logging.FileHandler;
import java.util.logging.Logger;
import java.util.logging.SimpleFormatter;

public class LoggingAPI {

    private static final Logger logger =
        Logger.getLogger(LoggingAPI.class.getName());

    public static void main(String[] args) {

        try {
            FileHandler fileHandler =
                new FileHandler("application.log");

            fileHandler.setFormatter(
                new SimpleFormatter()
            );

            logger.addHandler(fileHandler);

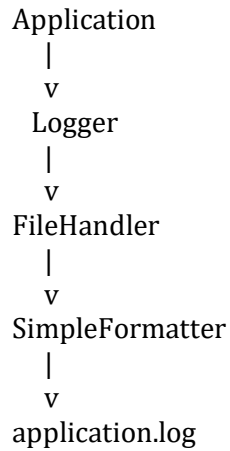
            logger.info("Application started.");
            logger.warning("Warning message.");
            logger.severe("Error message.");

            fileHandler.close();

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Flow



7. Logging an Exception

A major advantage of the Logging API is that an exception can be logged together with its stack trace.

```
import java.util.logging.Level;
import java.util.logging.Logger;

public class LoggingAPI {

    private static final Logger logger =
        Logger.getLogger(LoggingAPI.class.getName());

    public static void main(String[] args) {

        try {

            int result = 10 / 0;

            System.out.println(result);

        } catch (ArithmeticException e) {

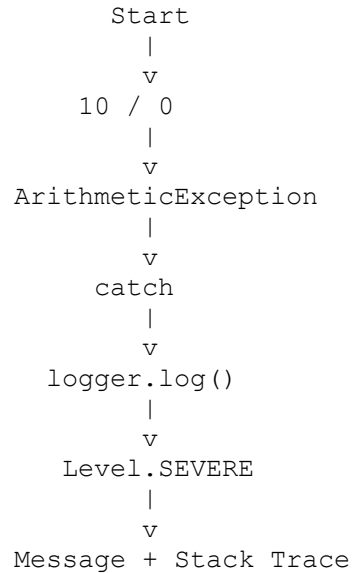
            logger.log(
                Level.SEVERE,
                "An arithmetic error occurred.",
                e
            );

        }

    }

}
```

Flowchart



8. Important Classes in java.util.logging

Class	Purpose
Logger	Main API for creating log messages
Level	Defines logging severity levels
Handler	Sends log records to a destination
ConsoleHandler	Sends logs to the console
FileHandler	Sends logs to a file
Formatter	Formats log records
SimpleFormatter	Provides a simple text format
LogRecord	Represents an individual log event
Filter	Controls which log records are processed

9. System.out.println() vs Logging API

Traditional approach

```
System.out.println("User logged in");
```

Logging approach

```
logger.info("User logged in");
```

Logging provides additional capabilities such as:

```
Logging API
|
+--> Severity Levels
|
+--> Timestamps
|
+--> Class/Method Information
|
+--> Console Output
|
+--> File Output
|
+--> Exception Stack Traces
|
+--> Configurable Filtering
```

10. Key Points

1. The Java Logging API was introduced in **Java 1.4**.
2. It is located in the **java.util.logging** package.
3. Logger is the central logging class.
4. Level defines the severity of a message.
5. Handler determines where log messages are sent.
6. Formatter controls the output format.
7. Logs can be written to the **console or files**.
8. Exceptions can be logged with their complete stack traces.
9. Logging is more flexible and maintainable than scattering `System.out.println()` throughout an application.

In one line

Java Logging API = Logger + Level + Handler + Formatter → structured and configurable application logging.

Java 1.4 — NIO (New I/O)

Java Version: 1.4

Year: 2002

Feature: NIO

UniqueClassName: NIOBasics

1. Definition

NIO (New I/O) is a Java I/O framework introduced in **Java 1.4** to provide more efficient and flexible mechanisms for **file handling, buffering, and non-blocking network I/O**.

The core NIO APIs are primarily located in:

java.nio

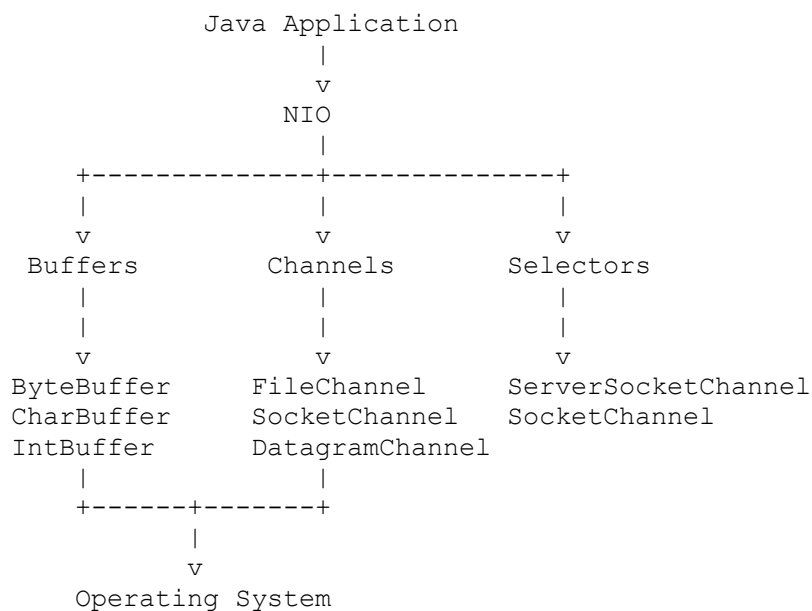
java.nio.channels

java.nio.charset

Unlike traditional stream-based I/O, NIO introduced concepts such as:

- **Buffers**
- **Channels**
- **Selectors**
- **Non-blocking I/O**
- **Character sets**
- **File mapping**

2. NIO Architecture

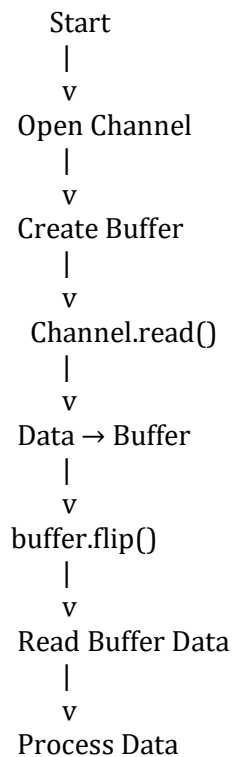


|
v
File / Network / Device

3. Traditional I/O vs NIO

Traditional I/O	NIO
Stream-oriented	Buffer-oriented
InputStream / OutputStream	Buffer / Channel
Usually blocking	Supports non-blocking I/O
Sequential stream processing	Buffer-based processing
Thread-per-connection commonly used	Selectors can manage multiple channels
Older I/O model	Introduced in Java 1.4

4. NIO Flowchart



```
|  
v  
buffer.clear()  
|  
v  
End
```

The basic NIO pattern is:

```
Channel  
|  
| read()  
v  
Buffer  
|  
| flip()  
v  
Read / Process  
|  
| clear()  
v  
Ready for next operation
```

5. Important NIO Components

A. Buffer

A **Buffer** is a container for data that can be read from or written to a channel.

Common buffers include:

```
ByteBuffer  
CharBuffer  
ShortBuffer  
IntBuffer  
LongBuffer  
FloatBuffer  
DoubleBuffer
```

The most commonly used is:

```
ByteBuffer
```

B. Channel

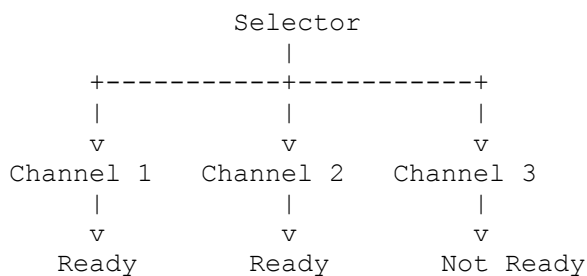
A **Channel** represents a connection through which data can be read or written.

Common channels include:

Channel	Purpose
FileChannel	File I/O
SocketChannel	TCP client networking
ServerSocketChannel	TCP server networking
DatagramChannel	UDP networking

C. Selector

A **Selector** allows a single thread to monitor multiple selectable channels.



This was particularly important for scalable network applications.

6. Java Code Example — ByteBuffer

```
import java.nio.ByteBuffer;

public class NIOBasics {

    public static void main(String[] args) {

        // Create a buffer
        ByteBuffer buffer = ByteBuffer.allocate(20);

        // Write data into the buffer
        buffer.put("Hello Java".getBytes());

        // Switch from writing mode to reading mode
        buffer.flip();

        // Read data from the buffer
```

```

        while (buffer.hasRemaining()) {
            System.out.print((char) buffer.get());
        }

        // Prepare buffer for another write operation
        buffer.clear();
    }
}

```

Output

Hello Java

7. Understanding ByteBuffer

The most important buffer operations are:

```

allocate()
|
v
put()
|
v
flip()
|
v
get()
|
v
clear()

```

Meaning

Method	Purpose
allocate()	Creates a buffer
put()	Writes data into the buffer
flip()	Switches buffer from writing to reading
get()	Reads data from the buffer
clear()	Resets buffer for another write

8. Java Code Example — FileChannel

NIO can also read files using FileChannel.

```
import java.io.FileInputStream;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;

public class NIOBasics {

    public static void main(String[] args) {

        try (FileInputStream input =
            new FileInputStream("data.txt");
            FileChannel channel =
                input.getChannel()) {

            ByteBuffer buffer =
                ByteBuffer.allocate(1024);

            while (channel.read(buffer) != -1) {

                buffer.flip();

                while (buffer.hasRemaining()) {
                    System.out.print(
                        (char) buffer.get()
                    );
                }

                buffer.clear();
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

File reading flow

```
data.txt
|
v
FileInputStream
|
v
FileChannel
|
| read()
v
ByteBuffer
|
```

| flip()
v
Read Bytes
|
v
Java Application

9. NIO File Copy Example

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.nio.channels.FileChannel;

public class NIOBasics {

    public static void main(String[] args) {

        try (
            FileInputStream source =
                new FileInputStream("source.txt");

            FileOutputStream destination =
                new FileOutputStream("copy.txt");

            FileChannel sourceChannel =
                source.getChannel();

            FileChannel destinationChannel =
                destination.getChannel()
        ) {

            destinationChannel.transferFrom(
                sourceChannel,
                0,
                sourceChannel.size()
            );

            System.out.println(
                "File copied successfully."
            );

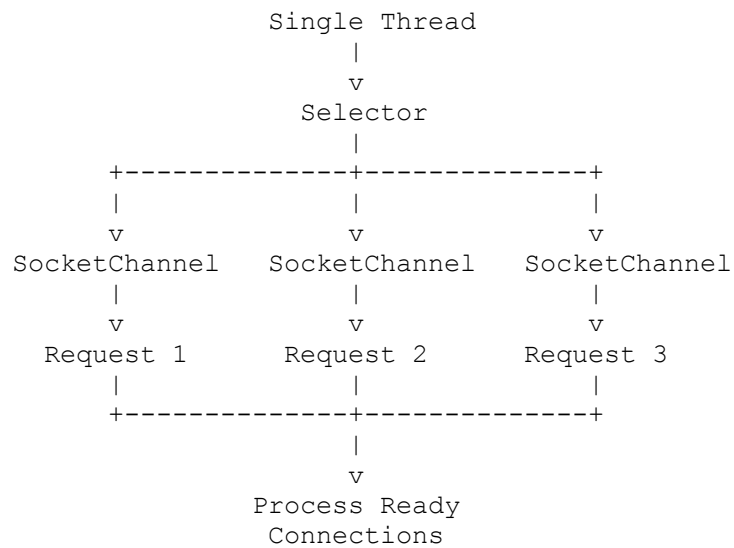
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Flow

```
source.txt
|
v
Source FileChannel
|
v
transferFrom()
|
v
Destination FileChannel
|
v
copy.txt
```

10. NIO Selector Concept

One of the most important features of NIO was **non-blocking I/O with selectors**.



Instead of requiring one thread to wait for each connection, a selector can monitor multiple channels and report which channels are ready for I/O.

11. Key NIO Classes

Package	Important Classes
java.nio	Buffer, ByteBuffer, CharBuffer
java.nio.channels	Channel, FileChannel, SocketChannel, ServerSocketChannel, Selector
java.nio.charset	Charset, CharsetDecoder, CharsetEncoder

12. Key Points

1. **NIO was introduced in Java 1.4.**
2. NIO means **New I/O**.
3. It introduced the **Buffer-Channel model**.
4. ByteBuffer is one of its most commonly used classes.
5. FileChannel provides efficient file operations.
6. SocketChannel and ServerSocketChannel support network I/O.
7. Selector enables multiplexed, non-blocking I/O.
8. NIO is useful for high-performance file and network applications.
9. flip(), get(), put(), and clear() are fundamental buffer operations.

In one line

Java NIO = Buffers + Channels + Selectors → flexible and efficient file and network I/O.

Java 1.4 — XML APIs (JAXP)

Java Version: 1.4

Year: 2002

Feature: XML APIs (JAXP)

UniqueClassName: XML_JAXP

1. Definition

JAXP (Java API for XML Processing) is a standard Java API for **processing XML documents**.

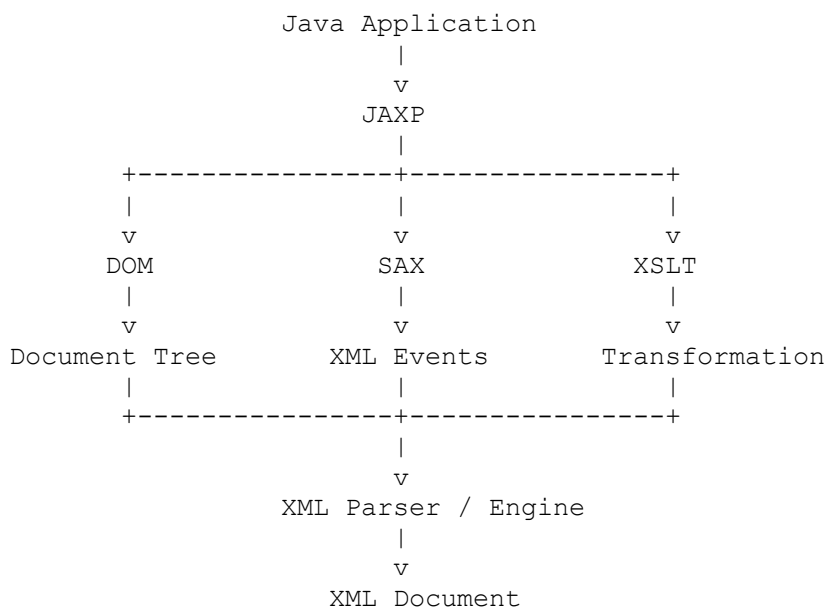
JAXP provides a common interface for working with XML without requiring application code to depend directly on a particular XML parser implementation.

JAXP supports major XML-processing approaches such as:

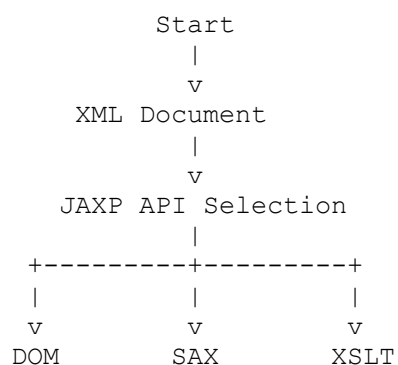
- **DOM** — loads the XML document into an in-memory tree.
- **SAX** — processes XML sequentially through events.
- **XSLT** — transforms XML documents using stylesheets.

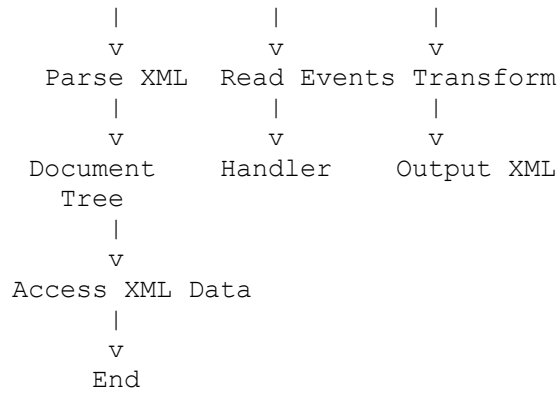
In Java 1.4, JAXP became an important standard mechanism for XML processing in the Java platform.

2. JAXP Architecture



3. JAXP Flowchart





4. DOM Processing

DOM (Document Object Model) reads an XML document and creates an in-memory tree representing the document.

For example:

```

<student>
  <name>Anurag</name>
  <age>25</age>
</student>
  
```

becomes conceptually:

```

Document
|
+-- student
|
|   +-- name
|   |   |
|   |   +-- Anurag
|   |
|   +-- age
|       |
|       +-- 25
  
```

DOM flow

```

XML File
|
v
DocumentBuilderFactory
|
v
DocumentBuilder
|
v
  
```

```
parse()
  |
  v
DOM Document
  |
  v
Element / Node
  |
  v
Read XML Data
```

5. Java Code Example — JAXP DOM

```
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;

import org.w3c.dom.Document;
import org.w3c.dom.Element;

public class XML_JAXP {

    public static void main(String[] args) {

        try {
            // Create a JAXP factory
            DocumentBuilderFactory factory =
                DocumentBuilderFactory.newInstance();

            // Create a document builder
            DocumentBuilder builder =
                factory.newDocumentBuilder();

            // Parse XML file
            Document document =
                builder.parse("student.xml");

            // Get the root element
            Element root =
                document.getDocumentElement();

            System.out.println(
                "Root Element: " +
                root.getNodeName()
            );

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Suppose student.xml contains:

```
<?xml version="1.0"?>
<student>
    <name>Anurag</name>
    <age>25</age>
</student>
```

Output

Root Element: student

6. Reading XML Elements

```
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;

import org.w3c.dom.Document;
import org.w3c.dom.NodeList;

public class XML_JAXP {

    public static void main(String[] args) {

        try {
            DocumentBuilderFactory factory =
                DocumentBuilderFactory.newInstance();

            DocumentBuilder builder =
                factory.newDocumentBuilder();

            Document document =
                builder.parse("student.xml");

            NodeList names =
                document.getElementsByTagName("name");

            NodeList ages =
                document.getElementsByTagName("age");

            System.out.println(
                "Name: " +
                names.item(0).getTextContent()
            );

            System.out.println(
                "Age: " +
                ages.item(0).getTextContent()
            );

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
    }  
  }  
}
```

Output

Name: Anurag
Age: 25

7. SAX Processing

SAX (Simple API for XML) processes an XML document sequentially instead of building the entire document tree in memory.

```
XML Document  
  |  
  v  
SAX Parser  
  |  
  +---- startElement()  
  |  
  +---- characters()  
  |  
  +---- endElement()  
  |  
  v  
Application
```

SAX flow

```
Start XML  
  |  
  v  
<student>  
  |  
  v  
startElement()  
  |  
  v  
<name>  
  |  
  v  
characters()  
  |  
  v  
</name>  
  |  
  v
```

```

endElement()
|
v
</student>
|
v
End XML

```

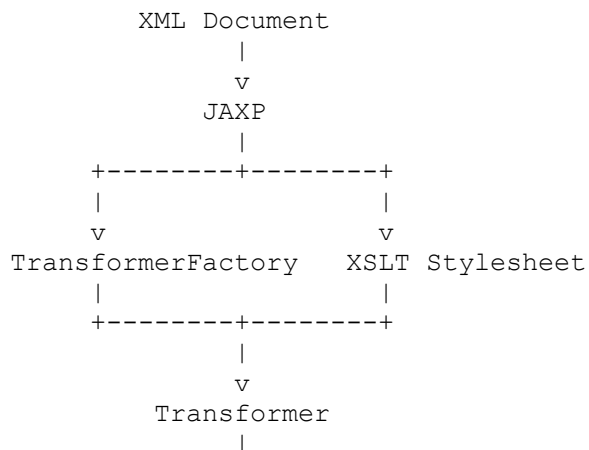
SAX is especially useful when processing **large XML documents** where loading the entire document into memory is undesirable.

8. DOM vs SAX

Feature	DOM	SAX
Processing	Tree-based	Event-based
Memory usage	Higher	Generally lower
Access	Random access to tree	Sequential
Modification	Convenient	Not designed for tree modification
Large XML	Can require substantial memory	Often better suited
API style	Document, Element, Node	Handler callbacks

9. JAXP + XSLT

JAXP also provides APIs for XML transformation using **XSLT**.



v
Transformed Output

The relevant classes are in:

javax.xml.transform

For example:

TransformerFactory

Transformer

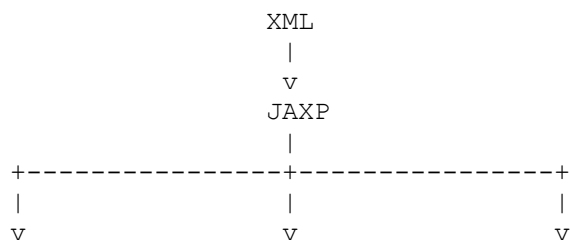
Source

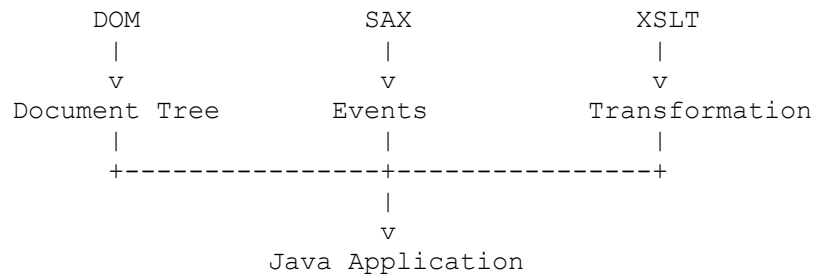
Result

10. Important JAXP APIs

API / Class	Purpose
DocumentBuilderFactory	Creates DOM document builders
DocumentBuilder	Parses XML into a DOM document
SAXParserFactory	Creates SAX parser objects
SAXParser	Performs SAX parsing
TransformerFactory	Creates XSLT transformers
Transformer	Performs XML transformations
Source	Represents transformation input
Result	Represents transformation output

11. Overall JAXP Flow





Key Points

1. **JAXP** stands for **Java API for XML Processing**.
2. It provides standard APIs for processing XML.
3. It supports **DOM, SAX, and XSLT** processing.
4. DOM represents XML as an in-memory tree.
5. SAX processes XML sequentially using events.
6. XSLT transforms XML into another form.
7. JAXP separates application code from the underlying XML parser implementation.
8. Important packages include `javax.xml.parsers`, `org.w3c.dom`, `org.xml.sax`, and `javax.xml.transform`.

In one line

JAXP = Standard Java XML APIs → DOM + SAX + XSLT → XML parsing, processing, and transformation.

Java 1.4 — Preferences API

Java Version: 1.4

Year: 2002

Feature: Preferences API

UniqueClassName: PreferencesAPI

1. Definition

The **Java Preferences API** provides a standard mechanism for storing and retrieving **user preferences and application configuration values**.

It was introduced in **Java 1.4** in the:

`java.util.prefs`

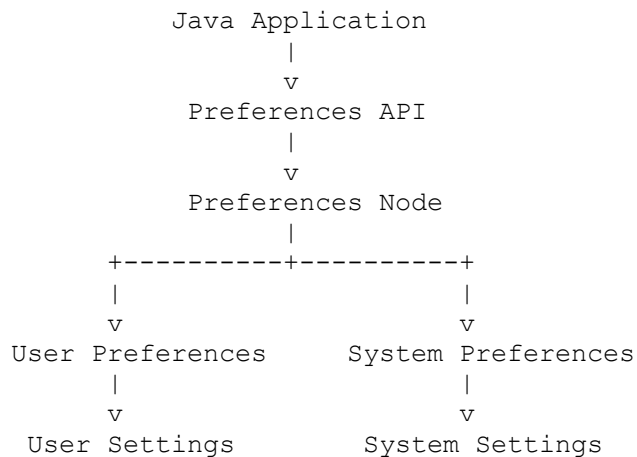
package.

Instead of hard-coding configuration values or managing platform-specific configuration files directly, an application can use Preferences to store simple key-value pairs.

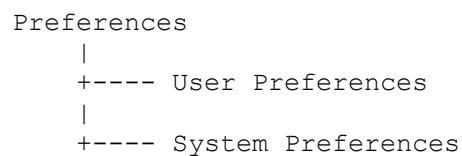
Typical examples include:

- User interface settings
- Window size and position
- User-selected options
- Application configuration
- Last-used directory
- User-specific settings

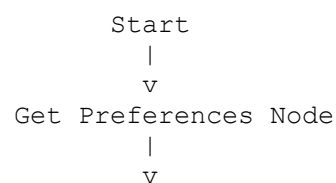
2. Preferences API Architecture

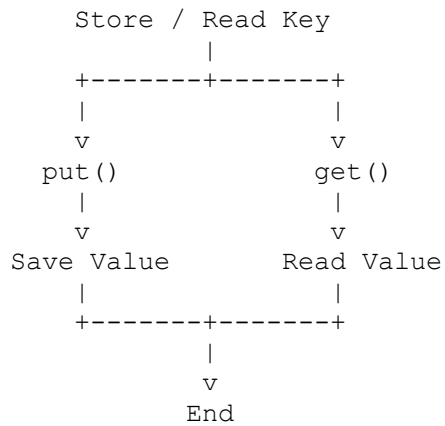


The API provides two main preference trees:



3. Preferences Flowchart





4. Basic Java Code Example

```
import java.util.prefs.Preferences;

public class PreferencesAPI {

    public static void main(String[] args) {

        // Get preferences associated with this class
        Preferences preferences =
            Preferences.userNodeForPackage(
                PreferencesAPI.class
            );

        // Store preferences
        preferences.put("username", "Anurag");
        preferences.put("language", "English");

        // Read preferences
        String username =
            preferences.get("username", "Unknown");

        String language =
            preferences.get("language", "Unknown");

        System.out.println(
            "Username: " + username
        );

        System.out.println(
            "Language: " + language
        );

    }
}
```

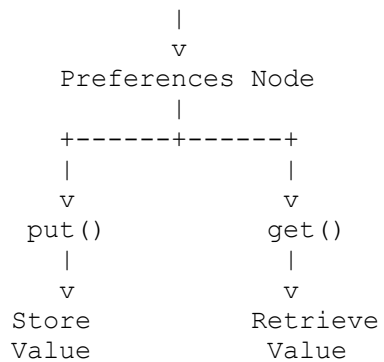
Output

Username: Anurag
Language: English

5. How Preferences Works

The basic pattern is:

```
Preferences.userNodeForPackage()
```



For example:

```
preferences.put("theme", "dark");
```

stores:

Key = theme
Value = dark

Then:

```
String theme = preferences.get("theme", "light");
```

retrieves the value.

6. Different Data Types

The Preferences API supports more than strings.

String

```
preferences.put("name", "Anurag");

String name =
    preferences.get("name", "Unknown");
```

Integer

```
preferences.putInt("age", 25);

int age =
    preferences.getInt("age", 0);
```

Boolean

```
preferences.putBoolean("darkMode", true);

boolean darkMode =
    preferences.getBoolean("darkMode", false);
```

Double

```
preferences.putDouble("price", 99.50);

double price =
    preferences.getDouble("price", 0.0);
```

Long

```
preferences.putLong("timestamp", 123456789L);

long timestamp =
    preferences.getLong("timestamp", 0L);
```

7. Complete Example with Multiple Preferences

```
import java.util.prefs.Preferences;

public class PreferencesAPI {

    public static void main(String[] args) {

        Preferences preferences =
            Preferences.userNodeForPackage(
```

```
        PreferencesAPI.class
    );

    // Store preferences
    preferences.put("username", "Anurag");
    preferences.put("language", "English");
    preferences.putInt("fontSize", 16);
    preferences.putBoolean("darkMode", true);

    // Retrieve preferences
    String username =
        preferences.get("username", "Unknown");

    String language =
        preferences.get("language", "Unknown");

    int fontSize =
        preferences.getInt("fontSize", 12);

    boolean darkMode =
        preferences.getBoolean(
            "darkMode",
            false
        );

    System.out.println(
        "Username: " + username
    );

    System.out.println(
        "Language: " + language
    );

    System.out.println(
        "Font Size: " + fontSize
    );

    System.out.println(
        "Dark Mode: " + darkMode
    );
}
}
```

Output

Username:	Anurag
Language:	English
Font Size:	16
Dark Mode:	true

8. User Preferences vs System Preferences

The API provides two important methods.

User Preferences

```
Preferences.userNodeForPackage(  
    PreferencesAPI.class  
);
```

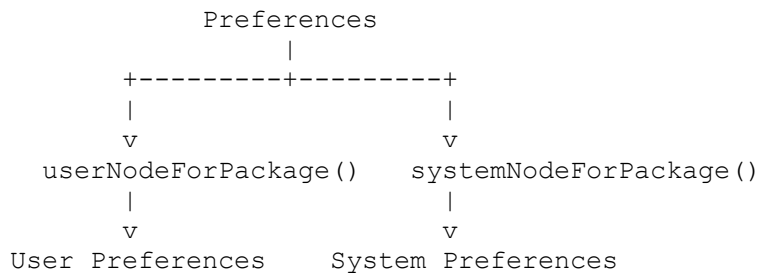
Used for preferences associated with the current user.

System Preferences

```
Preferences.systemNodeForPackage(  
    PreferencesAPI.class  
);
```

Used for preferences associated with the system/application environment.

Flowchart



9. Removing a Preference

A particular preference can be removed using `remove()`:

```
preferences.remove("username");
```

You can also remove all preferences in a node:

```
preferences.clear();
```

Example:

```
import java.util.prefs.Preferences;  
  
public class PreferencesAPI {
```

```

public static void main(String[] args)
    throws Exception {

    Preferences preferences =
        Preferences.userNodeForPackage(
            PreferencesAPI.class
        );

    preferences.put("username", "Anurag");

    System.out.println(
        preferences.get("username", "Unknown")
    );

    preferences.remove("username");

    System.out.println(
        preferences.get("username", "Unknown")
    );
}
}

```

Output:

Anurag
Unknown

10. Important Preferences Methods

Method	Purpose
userNodeForPackage()	Gets user preference node
systemNodeForPackage()	Gets system preference node
put()	Stores a String
get()	Retrieves a String
putInt()	Stores an integer
getInt()	Retrieves an integer
putBoolean()	Stores a boolean
getBoolean()	Retrieves a boolean
putLong()	Stores a long

getLong()	Retrieves a long
putDouble()	Stores a double
getDouble()	Retrieves a double
remove()	Removes a preference
clear()	Removes preferences from a node
keys()	Returns preference keys
node()	Creates/accesses a child node

11. Preferences API vs Properties

Preferences API	Properties
Designed for user/system preferences	General key-value configuration
Platform-independent API	Usually associated with files/streams
Supports user and system preference trees	No built-in user/system tree
Provides typed getters/setters	Primarily String-based
Can persist preferences through platform-specific mechanisms	Commonly persisted manually

12. Key Points

1. **Preferences API was introduced in Java 1.4.**
2. It is located in **java.util.prefs**.
3. The central class is Preferences.
4. It stores configuration as **key-value pairs**.
5. It supports **user preferences** and **system preferences**.
6. It supports values such as String, int, boolean, long, and double.
7. **put()** stores values and **get()** retrieves them.
8. **remove()** deletes a particular preference.
9. **clear()** removes preferences from a node.
10. It is useful for persistent application/user settings.

In one line

Java Preferences API = Preferences nodes + key-value storage → simple, persistent user and application preferences.